

פרק 1 · RED TEAM GUIDES

יסודות מערכות הפעלה

מדריך מלא למתחילים — בעברית

Red Team Guides Series · 2026

פרק 1 — יסודות מערכות הפעלה

מה זה בכלל מערכת הפעלה?

בוא נתחיל מהשאלה הכי בסיסית: מה קורה כשאתה לוחץ על כפתור ההפעלה של המחשב?

החומרה — המעבד, הזיכרון, הדיסק — לא יודעת לבד מה לעשות. היא רק מכונה. כדי שהמכונה הזאת תהפוך למשהו שימושי — שתוכל לפתוח דפדפן, לכתוב מסמך, לשמוע מוזיקה — צריך תוכנה שתשמש כ"מתרגם" בין האדם לחומרה. התוכנה הזאת היא **מערכת ההפעלה** (Operating System, בקצרה: OS).

תחשוב על זה ככה: החומרה היא מנוע הרכב, ומערכת ההפעלה היא לוח הבקרה, ההגה, הדוושות. בלי לוח הבקרה, המנוע לא שווה הרבה לנהג הממוצע.

מערכת ההפעלה אחראית על:

- ניהול משאבים — מי מקבל כמה זיכרון RAM, כמה זמן מעבד, גישה לדיסק
- תקשורת עם חומרה — להגיד לכרטיס המסך מה להציג, לכרטיס הרשת מה לשלוח
- ניהול קבצים — לשמור, לקרוא, למחוק קבצים על הדיסק
- ניהול משתמשים — מי מורשה לעשות מה
- הרצת תוכניות — לאפשר לתוכנות לרוץ בו-זמנית בלי שיתנגשו

קצת היסטוריה (רק מה שחשוב להבין)

בשנות ה-50 וה-60 לא היו מערכות הפעלה. כתבת תוכנית, העמסת אותה על הזמן של המחשב (שהיה גדול כמו חדר), והמחשב הריץ רק אותה. אף אחד אחר לא יכול היה להשתמש בו בינתיים.

אחר כך הגיע הרעיון של **זמן שיתוף (Time-sharing)** — הרעיון שמחשב אחד יכול לשרת הרבה משתמשים בו-זמנית, על ידי כך שהוא מחלק את זמן המעבד בין כולם במהירות כל כך גבוהה שאף אחד לא מרגיש.

משם התפתחו **Unix** בשנות ה-70 — מערכת שהניחה את הרוב המוחלט של הבסיס שעליו אנחנו עובדים היום. לינוקס, macOS — כולם צאצאים של Unix.

Windows לעומת זאת הגיע מכיוון אחר לגמרי — **DOS (Disk Operating System)** של IBM ו-Microsoft, ועם השנים התפתח לעצמאות.

למה זה חשוב? כי כשתבין את ההיסטוריה תבין **למה** הדברים עובדים כפי שהם עובדים.

שלושת השחקנים הגדולים

Windows

- המערכת הנפוצה ביותר בשולחנות עבודה ובארגונים
- ממשק גרפי (GUI) כבסיס
- קבצי הגדרות ב-Registry (מסד נתונים מרכזי לכל הגדרות המערכת)
- מנגנון הרשאות מבוסס על ACL (Access Control Lists)
- Active Directory — מערכת ניהול משתמשים ארגונית (נלמד בהמשך)
- קבצי הפעלה בסיומת `.exe`, ספריות בסיומת `.dll`

Linux

- מערכת קוד פתוח, בחינם
- שולטת בשרתים, ענן, מכשירי IoT, ו-90% מהאינטרנט
- כל דבר הוא קובץ — זה עיקרון יסוד של Linux
- ניהול דרך command line (שורת פקודה) הוא לב המערכת
- הרשאות פשוטות ואלגנטיות (owner, group, others)
- אלפי "הפצות" (distributions) — בגדול אותה מערכת, בגרסאות שונות

macOS

- מבוסס על Unix (Darwin)
- ממשק גרפי של Apple
- נפוץ אצל מפתחים ומעצבים
- משלב נוחות של GUI עם עוצמת Unix

בעולם Red Teaming: המטרות הן בד"כ Windows (ארגונים) ו-Linux (שרתים). הכלים שלנו ירוצו בד"כ על Linux (Kali, Parrot). לכן חייבים להכיר את שניהם.

הלב של המערכת: הקרנל (Kernel)

הקרנל הוא ה"גרעין" של מערכת ההפעלה — התוכנה שרצה ישירות על החומרה וקובעת חוקי משחק לכולם.

כשתוכנה רגילה (דפדפן, מחשבון) רוצה לעשות משהו — לקרוא קובץ, לשלוח מידע ברשת — היא **לא פונה ישירות לחומרה**. היא מבקשת מהקרנל לעשות זאת בשבילה. הקרנל בודק: "האם לתוכנה הזאת מותר?" ורק אז מבצע.

זה יוצר שני "עולמות":

- **Kernel Space** — שם הקרנל פועל. גישה מלאה לכל החומרה. טעות כאן = קריסת מערכת.
 - **User Space** — שם כל שאר התוכנות פועלות. מוגבלות. טעות כאן = התוכנה קורסת, לא המערכת.
- למה זה חשוב לנו?** כי הרבה מההתקפות המתקדמות (rootkits, privilege escalation) מנסות לעבור מ-User Space ל-Kernel Space. שם ההגנות נופלות.

תהליכים (Processes)

כשתוכנה רצה, היא נקראת **תהליך (Process)**. כל תהליך מקבל:

- **PID (Process ID)** — מספר זיהוי ייחודי
 - **מרחב זיכרון פרטי** — אחד לא יכול לקרוא של השני (בדרך כלל)
 - **הרשאות** — של המשתמש שהפעיל אותו
- כשאתה פותח את Chrome, הוא לא תהליך אחד — Chrome יוצר עשרות תהליכים (כרטיסייה אחת = תהליך נפרד). זה מכון — אם כרטיסייה אחת קורסת, השאר ממשיכות.

תהליכים ו-Threads

תהליך יכול להכיל כמה **threads** — "חוטים" שרצים בו-זמנית בתוך אותו תהליך. Thread הוא קל יותר ממשאבים מתהליך, וכולם חולקים את אותו מרחב זיכרון.

דוגמה: thread — Word אחד שומר אוטומטית, thread אחד בודק איות, thread אחד מגיב לקלט שלך. כולם חלק מאותו תהליך.

הורה וילד (Parent & Child)

בלינוקס, כל תהליך **נולד** מתהליך אחר. אם תריץ פקודה בטרמינל, הטרמינל יוצר תהליך-בן. כשהפקודה מסתיימת, הבן מת. התהליך הראשון שעולה עם המערכת הוא `init` (או `systemd` בלינוקס מודרני) — האב של כולם.

זיכרון (Memory)

RAM לעומת דיסק

- **RAM (זיכרון גישה אקראית)** — מהיר מאוד, מתאפס כשמכבים. כאן רצות תוכנות.
- **דיסק (Storage)** — איטי יחסית, שמור לצמיתות. כאן שמורים קבצים.

כשאתה פותח תוכנה, היא "עולה" מהדיסק ל-RAM. הקרנל מחליט איפה בדיוק ב-RAM היא תשכון.

זיכרון וירטואלי

כל תהליך "חושב" שיש לו את כל ה-RAM לעצמו. זה כמובן לא נכון — הקרנל משחק להם טריק. הוא נותן לכל תהליך **כתובות וירטואליות** ובסתר ממפה אותן לכתובות פיזיות אמיתיות ב-RAM.

זה מאפשר:

- בידוד בין תהליכים (כל אחד "חושב" שהוא לבד)
- להריץ תוכניות גדולות יותר מה-RAM הפיזי (חלק עובר לדיסק = Swap)
- ASLR — הגנת אבטחה שאנחנו נפגוש מאוחר יותר

Heap ו-Stack

בתוך הזיכרון של כל תהליך יש אזורים עם תפקידים שונים:

- **Stack** — זיכרון שמנוהל אוטומטית. כשפונקציה נקראת, "דוחפים" מידע ל-Stack. כשיוצאים, מורידים. מהיר אבל מוגבל בגודל.
- **Heap** — זיכרון שהתוכנה מבקשת בעצמה ("תן לי עוד 100MB"). גמיש אבל הבעלים אחראי לשחרר כשגמר.

למה זה חשוב לנו? Buffer Overflow — אחת ההתקפות הקלאסיות — מנצלת את ה-Stack. נלמד בהמשך.

מערכת הקבצים (File System)

מה זה בכלל קובץ?

קובץ הוא סדרה של bytes (בתים) ששמורה על הדיסק עם שם. הכל קובץ: תמונה, תוכנה, טקסט, סרטון — הכל bytes.

מערכות קבצים נפוצות

שם	איפה נפוץ	מה מיוחד בו
NTFS	Windows	הרשאות ACL, הצפנה (EFS), journal
ext4	Linux	מהיר, יציב, journaling
APFS	macOS	הצפנה מובנית, snapshots
FAT32	USB drives, חלקת boot	ישן, תואם לכולם
exFAT	USB drives גדולים	כמו FAT32 אבל עם קבצים גדולים

מבנה תיקיות — Linux

בלינוקס הכל מתחיל מ- / (נקרא "root" — שורש). כל שאר התיקיות הן ענפים ממנו:

```
/
├─ bin/      - תויסיסב תודוקפ (ls, cp, mv...)
├─ boot/     - ומצע לנרקה, לוחתא יצבק
├─ dev/      - (!צבוק לכה - תשר סיטרכ, מיקסיד) מירישכמ
├─ etc/      - תכרעמה לש תורדגה יצבק
├─ home/     - מישמתשמ לש תוישיא תויקית (/home/john, /home/sarah)
├─ lib/      - תופתושמ תוירפס
├─ media/    - USB/CD לש הניגע תודוקנ
├─ opt/      - ישילש-דצ תונכות
├─ proc/     - (קסידב אל, RAM-ב קר מייק) מיכילהת לע יח עדימ
├─ root/     - root שמתשמ לש תישיא הייקית
├─ sbin/     - (reboot, fsck...) תוילוהינ תודוקפ
├─ tmp/      - (שדחמ הלעפהב מיקחמנ) מיינמז מיצבק
├─ usr/      - מישמתשמ יבאשמו תונכות
|   └─ bin/   - ןהב שמתשת תודוקפה בור
|   └─ local/ - ןמצעב תנקתהש תונכות
|   └─ share/ - מיפתושמ מיצבק, דועית, icons
└─ var/      - logs, mail, databases: מינתשמ מינותנ
    └─ log/   - !מיגולה לכ ןאכ
```

```
c:\
├── Windows\          – תכרעמה יצבק
│   ├── System32\     – הביל תודוקפו תוירפס (64-bit)
│   ├── SysWOW64\     – (ירוססיה לובלב, מש) 32-bit תוירפס
│   └── Temp\         – מיינמז מיצבק
├── Program Files\    – תונקתומ תונכות (64-bit)
├── Program Files (x86)\ – תונקתומ תונכות (32-bit)
├── Users\           – מישמשמ תויקית
│   └── john\
│       ├── Desktop\
│       ├── Documents\
│       ├── Downloads\
│       └── AppData\  – תונכות תורדגה !רתסנ
└── ProgramData\     – תונכות לש מינות !רתסנ
```

טיפ **AppData** **Red Teaming**: **ProgramData** ו- הם מקומות זהב — שם תמצא cookies, passwords שמורים, הגדרות ש-AV לא תמיד בודק.

משתמשים והרשאות

הרעיון הבסיסי

לא כל משתמש יכול לעשות הכל. זה מכוון — **עיקרון ה-Least Privilege**: כל ישות (משתמש, תוכנה) צריכה רק את מה שהיא צריכה, לא יותר.

Linux — הרשאות

בלינוקס כל קובץ ותיקייה יש להם:

- **Owner** — הבעלים (משתמש ספציפי)
- **Group** — קבוצה שהקובץ שייך אליה
- **Others** — כולם השאר

לכל אחד מהשלושה יש שלוש הרשאות:

- **r (read)** — קריאה
- **w (write)** — כתיבה

• **x (execute)** — הרצה

כשתריץ `ls -l` תראה משהו כזה:

```
-rwxr-xr-- 1 john developers 4096 Jan 1 12:00 script.sh
```

נפרק אותו:

- **-** — סוג (קובץ רגיל; **d** = תיקייה; **l** = symbolic link)
- **rwx** — הבעלים (john): יכול לקרוא, לכתוב, להריץ
- **r-x** — הקבוצה (developers): יכולים לקרוא ולהריץ, לא לכתוב
- **r--** — כולם: רק לקרוא
- **john** — הבעלים
- **developers** — הקבוצה

chmod — שינוי הרשאות

```
chmod 755 script.sh    # rwxr-xr-x — מִטְפִּירְקֵסל צֹפֵן
chmod 644 file.txt     # rw-r--r-- — מִיִּצְבֵּקל צֹפֵן
chmod +x script.sh     # הצרה תאשרה פְסוּה
chmod -w file.txt      # הביתכ תאשרה רִסָה
```

מספרים: $r=4$, $w=2$, $x=1$. מחברים: $--rwx$, $6=rw-$, $5=r-x$, $4=r=7$

SUID, SGID, Sticky Bit — ההרשאות המיוחדות

אלה הרשאות מיוחדות שחשוב מאוד להבין:

SUID (Set User ID)

כשמוגדר על קובץ הפעלה, התוכנה רצה **עם הרשאות הבעלים שלה**, לא של מי שהפעיל אותה.

דוגמה: `passwd` (פקודה לשינוי סיסמה) שייכת ל-root ויש לה SUID. כששתמש רגיל מריץ אותה, היא יכולה לשנות את `/etc/shadow` שרק root יכול לגעת בו — אבל רק לשנות את הסיסמה של אותו משתמש.

למה זה מסוכן? כי אם תמצא תוכנה עם SUID שיש בה חור אבטחה — אפשר לנצל אותה כדי להריץ קוד כ-root!

```
find / -perm -4000 2>/dev/null # יצבק לכ תא אצמ SUID
```

:Sticky Bit

על תיקייה — רק הבעלים של קובץ יכול למחוק אותו, גם אם יש לאחרים הרשאת write לתיקייה. דוגמה: `/tmp` — כולם יכולים ליצור קבצים, אבל לא למחוק של אחרים.

Linux — sudo

`sudo` מאפשר למשתמש רגיל להריץ פקודות ספציפיות כ-root, בלי לדעת את סיסמת root.

```
sudo apt update      # כ-root 'apt update' צרה
sudo -l              # sudo? מע תושעל יל רתומ המ
sudo -u apache command # 'apache' שמתשמכ צרה
```

קובץ `/etc/sudoers` קובע מי יכול לעשות מה. **מצוין לטרגט ב-Red Teaming**.

Windows — הרשאות

Windows עובד אחרת. כל קובץ ותיקייה יש להם **ACL (Access Control List)** — רשימה מפורטת: "לגיון מותר לקרוא, לשרה מותר לקרוא ולכתוב, לקבוצה Developers מותר הכל".

סוגי חשבונות ב-Windows:

- **Administrator** — מלא שליטה
- **Standard User** — מוגבל
- **SYSTEM** — חזק יותר מ-Administrator! זה הקרנל בעצם
- **Local Service / Network Service** — לשירותי מערכת, הרשאות מוגבלות

UAC (User Account Control) — ה-popup שמופיע "האם אתה בטוח?". תפקידו לא לתת לתוכנות לקבל הרשאות Admin בלי ידיעתך. הרבה טכניקות Red Teaming עוסקות ב-UAC Bypass.

אתחול המערכת (Boot Process)

שלב 1: BIOS / UEFI

כשמדליקים את המחשב, המעבד קופץ לכתובת ידועה בזיכרון ה-Flash ומריץ קוד — **BIOS (Legacy)** או **UEFI** (מודרני).

UEFI/BIOS מבצע בדיקת חומרה (POST — Power-On Self Test), ואז מחפש מה להתחיל: דיסק?
USB? רשת?

UEFI הוא חשוב לנו כי:

- יש לו Secure Boot שמונע הפעלת קוד לא חתום
- ניתן לתקוף אותו (UEFI rootkits — הנמכרים ביותר)
- מאחסן הגדרות ב-NVRAM

שלב 2: Bootloader

UEFI/BIOS טוען את ה-Bootloader:

- GRUB בלינוקס — מציג תפריט, טוען את הקרנל
- Windows Boot Manager ב-Windows

שלב 3: הקרנל עולה

הקרנל עולה לזיכרון ומתחיל להתחיל:

- מזהה חומרה
- טוען drivers
- מגדיר זיכרון
- יוצר תהליכים ראשוניים

שלב 4: Init / systemd

הקרנל מפעיל את התהליך הראשון:

- systemd ברוב לינוקס מודרניות (PID 1)
- init בלינוקס ישנות
- smss.exe ב-Windows

מרגע זה, כל שירות ותהליך שעולה הוא צאצא של PID 1.

שלב 5: Login

המשתמש מתחבר — גרפי (Display Manager) או שורת פקודה (getty/login).

טיפ: Evil Maid attack — תוקף עם גישה פיזית יכול לשנות את ה-Bootloader כדי לגנוב סיסמאות.
Secure Boot נועד בדיוק למנוע זאת.

שורת הפקודה (Command Line Interface)

למה CLI?

ממשק גרפי יפה, אבל:

- מהיר יותר — פקודה אחת עושה מה שלחיצות גרפיות עושות בדקות
- ניתן לאוטומציה — לכתוב סקריפט שמריץ אלף פקודות
- נמצא בכל מקום — שרתים ללא מסך, חיבורים מרוחקים
- בהתקפות — לרוב מה שיש לנו גישה אליו הוא CLI בלבד

Linux — Bash

ה-Shell הנפוץ ביותר. `bash` הוא בעצמו תוכנה שמפרשת את מה שאתה כותב.

```

# טווינ
pwd                # Where Am I? – גצה
ls                 # List – גצה
ls -la            # מיטרפ + מירתסנ מיצבק ללזכ
cd /etc           # Change Directory
cd ..             # הלעמל תחא הייקית הלע
cd ~              # קלש תיבה תייקיתל רובע

# מיצבק
cp file.txt copy.txt    # Copy
mv file.txt /tmp/       # Move (מג Rename)
rm file.txt            # Remove – ויא Trash!
rm -rf /tmp/folder/     # (!תוריהז) הנכותו הייקית קחמ
mkdir newfolder        # Make Directory
touch newfile.txt      # קיר צבוק רוצ

# מיצבק תאירק
cat file.txt          # לכה גצה
less file.txt         # (תאצל q) הלילג מע גצה
head -n 20 file.txt   # תונושאר תורוש 20
tail -n 20 file.txt   # תונורחא תורוש 20
tail -f /var/log/auth.log # תמא נמזב צבוק ירחא בוקע

# שופיח
find /etc -name "*.conf"    # מש יפל מיצבק אצמ
grep "password" file.txt    # צבוקב תזורחמ שפח
grep -r "admin" /var/www/   # תיביסרוקר שפח
grep -i "error" /var/log/syslog # תונטק/תולודגל תושיגר אלל שופיח

# מיכילהתו מישמתשמ
whoami              # ?ינא ימ
id                  # ?ילש תוצובק, GID, ה-UID המ
ps aux              # מיצר מיכילהת גצה
ps aux | grep nginx # יפיצפס קילהת אצמ
kill 1234            # PID יפל קילהת גורה
kill -9 1234         # (SIGKILL) מויס הפכ
top                  # מיבאשמ תכירצו מיכילהת לש יח גצ

```

```
htop                # לש רתוי הפי הסרג

# תואשרה
sudo command        # כ-רור
su username          # Switch User
chmod 755 file       # תואשרה הנש
chown john:devs file # תולעב הנש

# (תותשר קרפב דמלנ – המדקמ) תשר
ifconfig             # (ןשי) תשר יקשממ גצה
ip addr              # (שדח) תשר יקשממ גצה
ping google.com      # רוביח קודב
netstat -tulpn       # מיחותפ מיטרופ גצה
ss -tulpn            # netstat לש השדח הסרג

# טלפ/טלק בותינ
echo "hello" > file.txt # (סרד) צבוקל בותכ
echo "hello" >> file.txt # צבוקל פסוה
cat file.txt | grep "error" # האבה הדוקפל טלקכ טלפ חלש – Pipe
command 2>/dev/null     # תואיגש קתשה
command 2>&1 | tee log.txt # צבוקל stderr וגו stdout מג רומש
```

PowerShell-ו Windows — CMD

Windows יש לו שניים:

CMD (Command Prompt) — הישן, מוגבל:

```
dir                # ls (ומכ) נכות גצה
cd C:\Users        # טווינ
copy file.txt dest\ # הקתעה
del file.txt        # הקיחמ
mkdir newfolder     # השדח הייקית
type file.txt        # צבוק גצה (cat ומכ)
findstr "password" *.txt # שופיח (grep ומכ)
tasklist            # מיכילהת גצה
taskkill /PID 1234 /F # רילהת גורה
ipconfig            # תשר גצה
netstat -ano        # מיחותפ מיטרופ
```

PowerShell — החדש, עוצמתי מאוד:

```
Get-ChildItem       # ls / dir
Set-Location C:\Users # cd
Copy-Item file.txt dest\ # cp
Remove-Item file.txt # rm
New-Item -Type Directory folder # mkdir
Get-Content file.txt # cat
Select-String "password" *.txt # grep
Get-Process         # ps
Stop-Process -Id 1234 -Force # kill
Get-NetIPAddress    # ifconfig
Get-NetTCPConnection # netstat

# תואמגוד – רתוי קזח PowerShell
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 # רתויב מישיתמה 10
Get-EventLog -LogName Security -Newest 100 # חטבא יגול
Invoke-WebRequest -Uri "http://example.com" -OutFile "file.html" # צבוק דרוה
```

PowerShell חשוב במיוחד ב-Red Teaming — הרבה מאוד התקפות מודרניות ב-Windows משתמשות בו.

Logs — יומני המערכת

הלוגים הם הזיכרון של המערכת. כל פעולה חשובה נרשמת. בתור Red Teamer, צריך לדעת:

1. מה מוקלט — כדי להבין מה יגלו עליך
2. איפה — כדי לנקות עקבות אחרי ביצוע

Linux — Logs

```
/var/log/
├─ syslog      – יללכ לכה
├─ auth.log    – !דואמ בושח ← sudo, SSH, תוסינכ
├─ kern.log    – לנרקה
├─ messages    – תויללכ תועדוה
├─ nginx/      – logs לש nginx
|   └─ access.log
|   └─ error.log
└─ apache2/    – logs לש Apache
```

```
tail -f /var/log/auth.log      # תמא ןמזב תוסינכ ירחא בוקע
grep "Failed password" /var/log/auth.log # תולשוכ תוסינכ
grep "sudo" /var/log/auth.log   # ישומיש sudo
journalctl -u ssh               # SSH תוריש לש סיגול (systemd)
journalctl --since "1 hour ago" # הנורחא העש
```

Windows — Event Logs

Windows מנהל לוגים בממשק גרפי (Event Viewer) ובפקודות:

לוגים חשובים:

- **Security** — כניסות, שינוי הרשאות, גישה לקבצים
- **System** — אירועי מערכת, שירותים
- **Application** — שגיאות תוכנות

Event IDs קריטיים לדעת:

משמעות	ID
כניסה מוצלחת	4624
כניסה כושלת	4625
ניסיון כניסה עם credentials מפורשים	4648
נוצר משתמש חדש	4720
נוסף למשתמש לקבוצת Administrators	4732
תהליך חדש נוצר	4688
שירות חדש הותקן	7045

```
Get-EventLog -LogName Security -InstanceId 4624 -Newest 50 # תוחלצום תוסינכ
Get-EventLog -LogName Security -InstanceId 4625 -Newest 50 # תולשוכ תוסינכ
wevtutil qe Security /c:10 /f:text /q:"*[System[EventID=4624]]" # CMD
```

משתני סביבה (Environment Variables)

משתני סביבה הם ערכים גלובליים שתוכנות יכולות לקרוא. חשוב מאוד!

Linux

```
env # לכה גצה
echo $PATH # תודוקפל שופיח יביתנ
echo $HOME # תיבה תייקית
echo $USER # שמתשמה םש
echo $SHELL # shell הזיא
export MY_VAR=value # שדח הנתשמ רדגה
```

PATH הוא קריטי. כשאתה מקליד `ls`, המערכת מחפשת קובץ בשם `ls` בכל תיקייה שב-PATH לפי הסדר. **PATH Hijacking** — טכניקת התקפה שמנצלת את זה.

```

set                # לכה גצה
echo %PATH%        # שופיח י'ביתנ
echo %USERNAME%    # שמתשמ שש
echo %COMPUTERNAME% # בשחמה שש
echo %APPDATA%     # AppData ת'יקית
set MY_VAR=value   # (ינמז) הנתשמ רדגה
setx MY_VAR value  # (עובק) הנתשמ רדגה

```

ב-Windows Variables, חשובות:

- `%APPDATA%` — `C:\Users\user\AppData\Roaming`
- `%LOCALAPPDATA%` — `C:\Users\user\AppData\Local`
- `%TEMP%` — תיקייה זמנית
- `%SYSTEMROOT%` — `C:\Windows`

Cron-ו Scheduled Tasks — משימות מתוזמנות

מנגנון שמריץ פקודות/סקריפטים בזמנים קבועים. מצוין למנגנוני Persistence (דרך להישאר בתוך מערכת גם אחרי הפעלה מחדש).

Linux — Cron

```

crontab -l        # יחכונה שמתשמה לש תונמזותמ תומישמ גצה
crontab -e        # קורע
crontab -l -u root # (תואשרה שורד) לש גצה root
cat /etc/crontab   # cron תמרב
ls /etc/cron.d/    # מ'פסונ cron 'צבק
ls /etc/cron.daily/ # ימוי מ'צרש מ'טפ'רקס

```

פורמט cron:

```
* * * * * command
| | | | |
| | | | └── עובשב מוי (0-7, 0=אר=0)
| | | └── שדוח (1-12)
| | └── שדוחב מוי (1-31)
| └── העש (0-23)
└── הקד (0-59)

0 2 * * * /usr/bin/backup.sh # ב-2:00 הליל לכ
*/5 * * * * /usr/bin/check.sh # תוקד 5 לכ
```

Windows — Task Scheduler

```
schtasks /query /fo LIST /v # תונמזותמה תומישמה לכ גצה
schtasks /create /tn "MyTask" /tr "C:\backdoor.exe" /sc onlogon # המישמ רוצ (Admin שורד)
```

שירותים (Services / Daemons)

שירות הוא תהליך שרץ ברקע, בלי ממשק גרפי, מתחיל עם המערכת.

Linux — systemd

```
systemctl list-units --type=service # מיתוריש לכ גצה
systemctl status ssh # תוריש בצמ SSH
systemctl start nginx # לעפה
systemctl stop nginx # רוצע
systemctl enable nginx # האבה הלעפהב מג לעפה
systemctl disable nginx # האבה הלעפהב ליעפת לא
systemctl is-enabled nginx # תולעל רדגומ סאה
```

שירות מוגדר בקובץ `.service`:

```
[Unit]
Description=My Backdoor
After=network.target

[Service]
ExecStart=/usr/bin/my-backdoor
Restart=always

[Install]
WantedBy=multi-user.target
```

Windows — Services

```
sc query # מיתוריש גצה
sc query state= all # מירוצע ללזכ
sc start servicename # לעפה
sc stop servicename # רוצע
sc create MyService binPath= "C:\evil.exe" start= auto # תוריש רוצ
```

```
Get-Service # לכה גצה
Get-Service -Name *sql* # שפח
Start-Service -Name wuauclt # לעפה
Set-Service -Name MyService -StartupType Automatic # יטמוטוא לעפה
```

שירותים הם מקום נפוץ ל-Persistence בהתקפות.

רשת ברמת OS (הקדמה)

נלמד לעומק בפרק רשתות, אבל כמה דברים בסיסיים עכשיו:

```
# Linux
ip addr show          # מיקשמת גצה
ip addr show eth0     # יפוצפס קשמת
ip route show         # בותינ תלבט

# Windows
ipconfig /all         # לכה גצה
route print            # בותינ תלבט
```

קבצי מפתח ב-Linux לרשת

```
/etc/hosts            - מוקמ יופימ: hostname → IP
/etc/resolv.conf      - יתרש DNS
/etc/network/interfaces - מיקשמת תורדגה (Debian/Ubuntu)
/etc/NetworkManager/ - תורדגה NetworkManager
```

חשוב מאוד — /etc/hosts

```
127.0.0.1    localhost
192.168.1.10  myserver.local
```

מה שכתוב כאן מנצח את ה-DNS. כלומר אם תוסיף:

```
1.2.3.4      google.com
```

... המחשב שלך יחשוב שגוגל נמצא ב-1.2.3.4. טכניקת התקפה: DNS Poisoning / Hosts file manipulation.

Linux

קובץ	מה יש שם
/etc/passwd	רשימת משתמשים (ללא סיסמאות!)
/etc/shadow	הסיסמאות המוצפנות — דרוש !root
/etc/group	רשימת קבוצות
/etc/sudoers	מי יכול sudo ומה
/etc/ssh/sshd_config	הגדרות שרת SSH
~/.ssh/authorized_keys	מפתחות SSH מורשים
~/.bash_history	היסטוריית פקודות!
~/.bashrc	סקריפט שרץ עם פתיחת bash
/etc/crontab	משימות מתוזמנות

Windows (Registry)

ה-Registry הוא מסד נתונים היררכי שמאחסן את כל הגדרות Windows. ניגש אליו דרך regedit או פקודות.

ענפים ראשיים:

- HKEY_LOCAL_MACHINE (HKLM) — הגדרות המחשב (כולם)
- HKEY_CURRENT_USER (HKCU) — הגדרות המשתמש הנוכחי
- HKEY_USERS (HKU) — הגדרות כל המשתמשים
- HKEY_CLASSES_ROOT (HKCR) — סוגי קבצים ותוכניות
- HKEY_CURRENT_CONFIG (HKCC) — הגדרות חומרה נוכחיות

Registry paths קריטיים:

HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	– מיִשְׁתַּשְׁמָה לִכּ Windows מֵע הַלּוֹע הֵם
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	– (יִחְכּוֹנָה שְׁמֵתִשְׁמָה) הַלּוֹע הֵם
HKLM\SYSTEM\CurrentControlSet\Services	– מִיִּתּוּרִישׁ
HKLM\SAM	– (!נִפְצוּם) תּוֹיִמוֹקֵם תּוֹאֲמָסִיס
HKLM\SECURITY\Policy\Secrets	– מִיִּנְסַחֲאֵם תּוֹדוּס

Registry = Persistence אהוב של תוקפים.

ציר האבטחה: CIA Triad

לפני שנסיים, מושג יסוד שצריך לדעת:

כל עניין אבטחת מידע סובב סביב שלושה עקרונות:

Confidentiality — C (סודיות): רק מורשים יכולים לגשת למידע

Integrity — I (שלמות): המידע לא שונה ללא אישור

Availability — A (זמינות): המידע זמין לשימוש כשצריך

בתור **Red Teamer**, תתמקד בלהפר את אחד (לרוב C ו-I). הארגון שמגן תמיד שואל: "מה תוקף יכול לעשות?"

סיכום — מה חשוב לזכור

1. **מערכת הפעלה** = שכבה בין החומרה לתוכנה. מנהלת הכל.
2. **Kernel Space vs User Space** — הגבול בין שליטה מלאה לשליטה מוגבלת.
3. **הכל קובץ בלינוקס** — כולל מכשירים, רשתות, תהליכים.
4. **הרשאות** — מי יכול מה. הרבה התקפות = עקיפת הרשאות.
5. **Logs** = עקבות. תדע מה מוקלט.
6. **Registry / etc** — שם מאוחסנות הגדרות. גם credentials.
7. **שירותים** + **Cron = persistence**. מריצים קוד אוטומטית.
8. **CLI** = כלי עיקרי. חייב להיות שגור בפה.

הפרק הבא: לינוקס בעומק — כל מה שצריך לדעת בשביל לעבוד ביעילות בסביבת **Red Teaming**.